

[딥러닝 파이토치 교과서] 6장 (1)

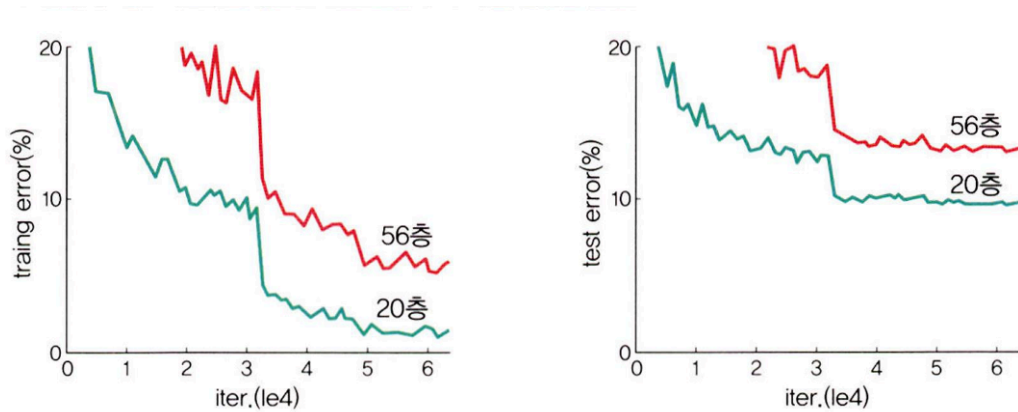
6.1.5 ResNet

- 정리

- 신경망은 깊이가 깊어질 수록 성능이 좋아지다 일정 단계에 다다르면 오히려 성능이 떨어짐

→ ResNet은 이를 해결하기 위해 레지듀얼 블록 도입

레지듀얼 블록은 기울기가 잘 전파될 수 있도록 일종의 숏컷을 만들어 기울기 소멸 문제 해결



- 블록

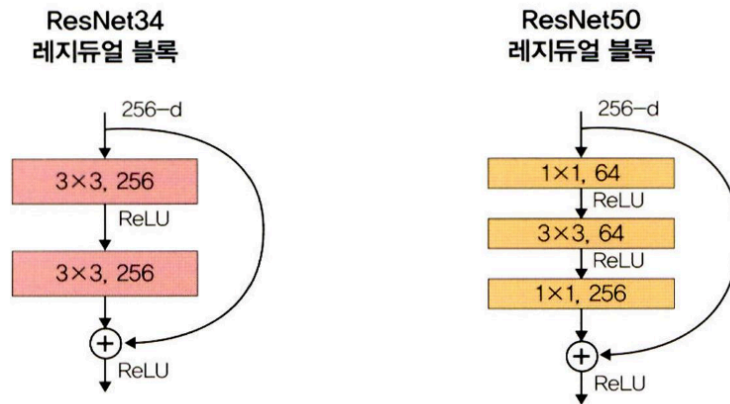
- 합성곱층을 하나의 블록으로 묶은 것
- 색상별로 묶인 블록의 계층들을 레지듀얼 블록이라고 하고, 레지듀얼 블록을 여러 개 쌓은 것을 ResNet이라고 함



- 하지만 이렇게 계층을 계속해서 쌓아 늘리면 파라미터 수가 문제가 됨 → 병목 블록을 둬

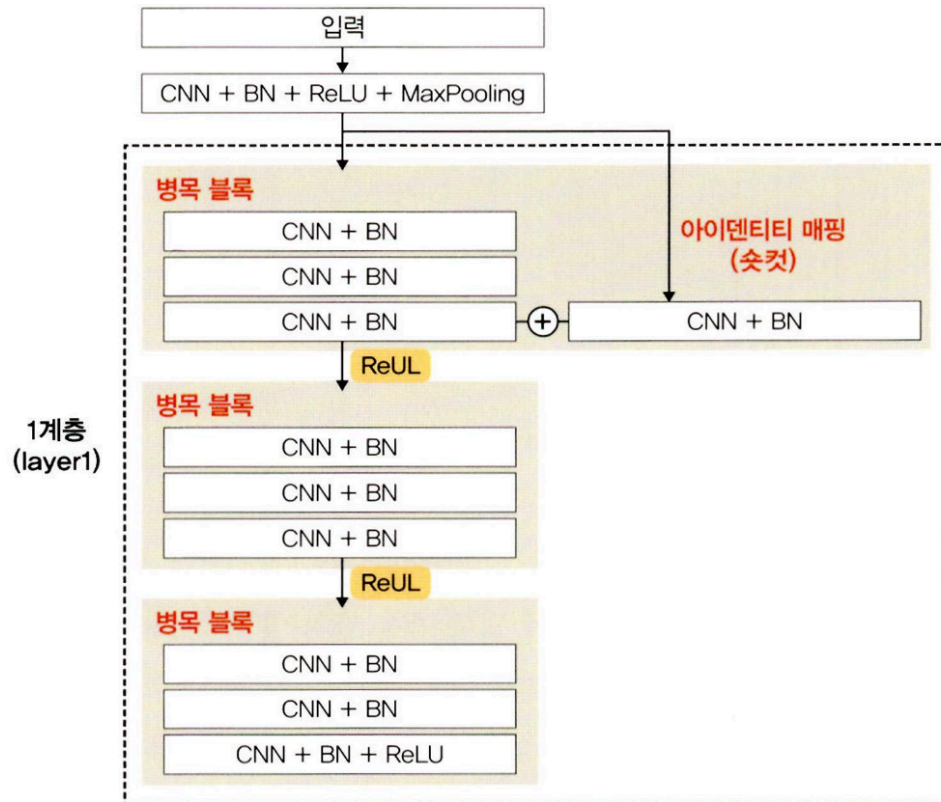
- 병목 블록

- 깊이가 깊어짐에도 파라미터를 감소하는 효과를 줌



기본 블록을 사용하는 ResNet과 병목 블록을 사용하는 ResNet

- 3×3 합성곱층 앞뒤로 1×1 합성곱층이 붙어 1×1 합성곱층의 채널 수를 조절하며 차원을 조절 가능함
→ 파라미터를 줄일 수 있음
- 아이덴티티 매핑(숏컷, 스킵 연결)
 - 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태로 출력되게 하는 것



그림의 + 기호가 있는 부분을 아이덴티티 매핑이라고 함

- 코드로 구현하면 다음과 같음

```
def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.bn2(x)

    if self.downsample is not None:
        i = self.downsample(i) // 다운샘플 적용

    x += i // 아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

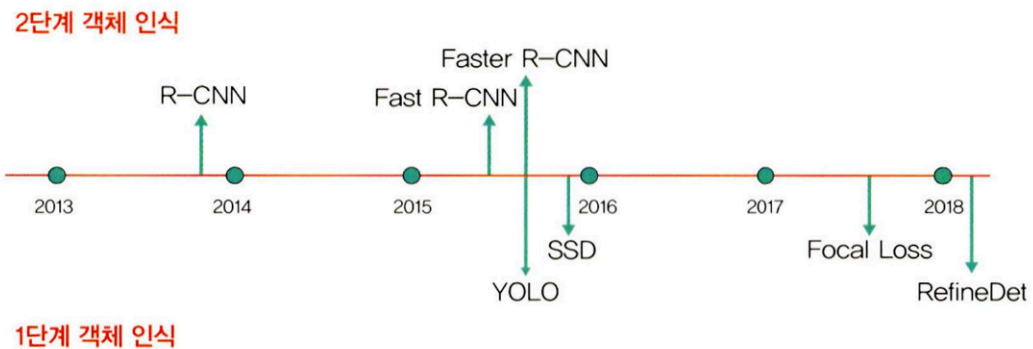
- 다운샘플
 - 특성맵 크기를 줄이기 위한 것, 풀링과 같은 역할
 - 형태가 다르면 아이덴티티 매핑을 할 수 없기 때문에, 다운샘플이 필요함

- 스트라이드 2를 가진 1x1 합성곱 계층을 연결해 다운샘플 수행

- 실습

6.2 객체 인식을 위한 신경망

- 객체인식이란?
 - 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술
 - 여러 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출
 - 딥러닝을 이용한 객체 인식 알고리즘은 크게 1단계 객체 인식과 2단계 객체 인식으로 나눌 수 있음



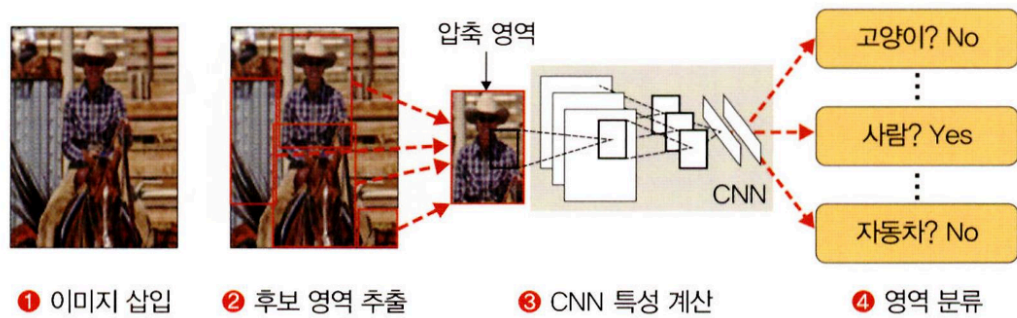
- 1단계 객체 인식
 - 분류와 위치 검출을 동시에 수행함
 - 속도가 빠르지만 정확도가 낮음
 - YOLO 계열과 SSD 계열
- 2단계 객체 인식
 - 분류와 위치 검출을 순차적으로 수행함
 - 속도가 느리지만 정확도가 높음
 - 대표적으로 CNN을 처음 적용시킨 R-CNN 계열

2단계 객체 인식 알고리즘에 대해 알아보기

6.2.1 R-CNN

- 정리

- 이미지 분류를 수행하는 CNN과, 객체가 있을 만한 영역을 제안해 주는 후보 영역 알고리즘을 결합한 알고리즘
- 학습 절차



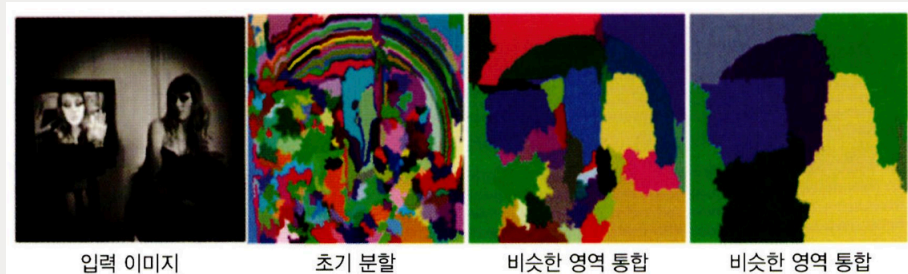
1. 이미지를 입력으로 받음
2. 2000개의 바운딩 박스를 선택적 알고리즘으로 추출한 후 잘라내고 227x227 픽셀로 통일
(CNN 모델에 넣기 위함)
3. 크기가 동일한 이미지 2000개 각각에 CNN 모델 적용
4. 각각 분류를 진행해 결과 도출

✓ 선택적 탐색

- 객체 인식이나 검출을 위한 가능한 후보 영역을 알아내는 방법
- 분할 방식을 이용해 시드(seed)를 선정하고, 그 시드에 대한 완전 탐색 적용
- 학습 과정
 - 1단계) 초기 영역 생성: 입력된 이미지를 영역 다수 개로 분할



- 2단계) 영역 여러 개로 나눈 것들을 비슷한 영역으로 통합
탐욕 알고리즘을 이용해 비슷한 영역이 하나로 통합될 때까지 반복



- 3단계) 통합된 이미지들을 기반으로 후보 영역(바운딩 박스) 추출

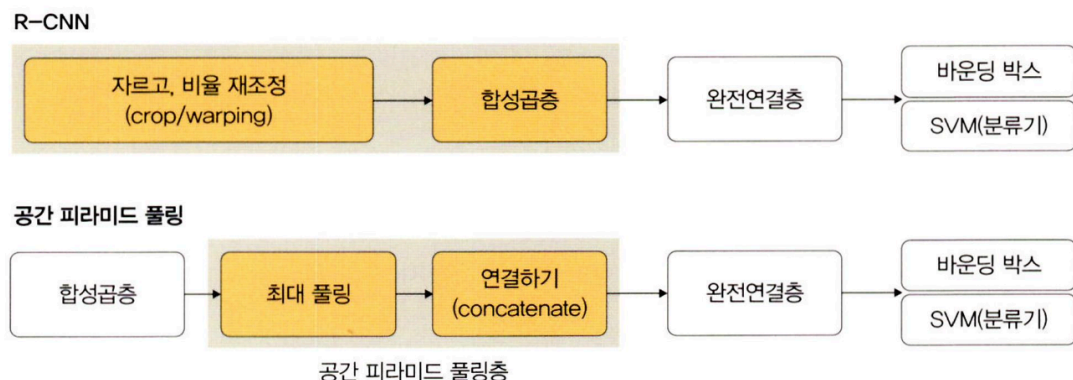


- 단점
 - 세 단계의 복잡한 과정

- 긴 학습 시간과 대용량 저장 공간
- 객체 검출 속도 문제
 - ⇒ 해결을 위해 Fast R-CNN 등장(6.2.3에서 추후 다룸)

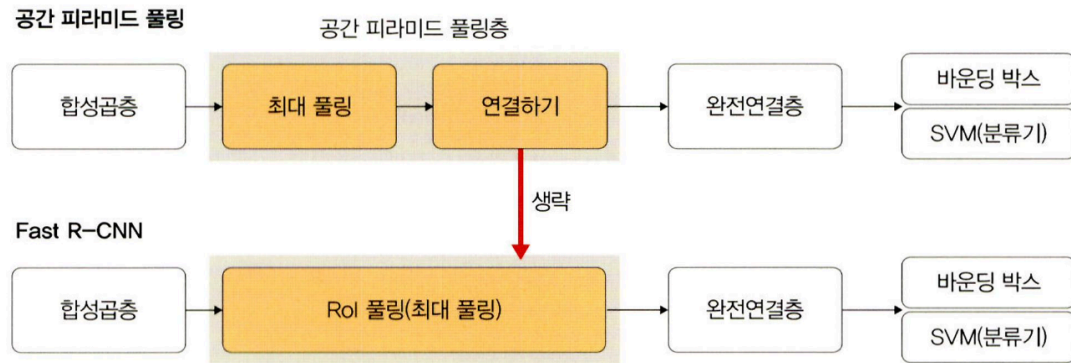
6.2.2 공간 피라미드 풀링

- 등장 배경
 - CNN 구조는 완전연결층을 위해 입력 이미지를 고정해야 했음
 - 물체의 일부분이 잘리거나 본래의 생김새와 달라지는 문제 발생
 - 이를 해결하기 위해 공간 피라미드 풀링 도입
- 정리
 - 입력 이미지의 크기에 관계없이 합성곱층을 통과시키고, 완전연결층에 전달되기 전 특성 맵들을 동일한 크기로 조절해주는 풀링층을 적용하는 기법
 - 원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있음
 - 이미지 분류나 객체 인식 같은 여러 작업에 적용할 수 있다는 장점!



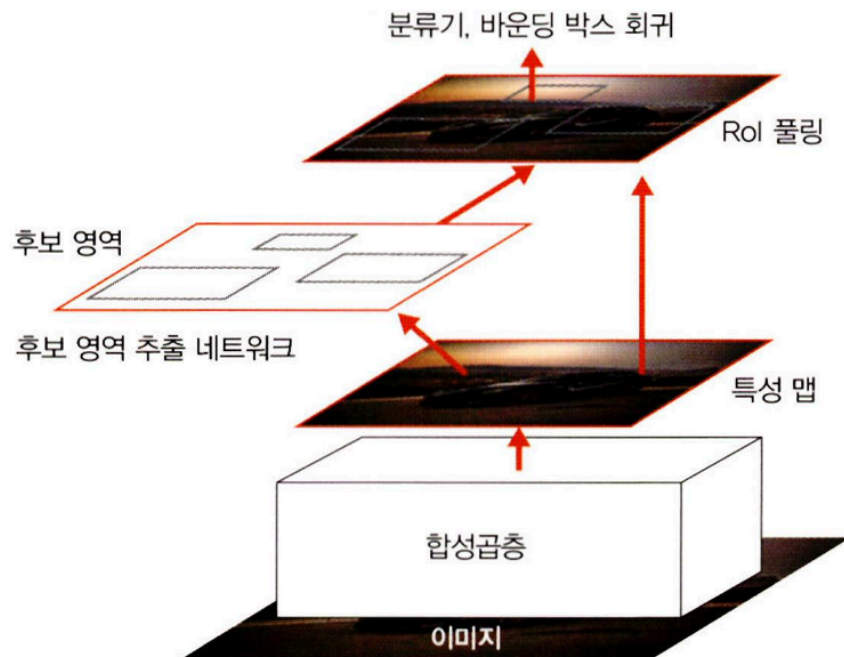
6.2.3 Fast R-CNN

- 정리
 - R-CNN의 속도 문제 개선을 위해 RoI 풀링 도입
 - *RoI 풀링: 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링 적용해 결과값 크기를 동일하게 맞추는 방법
 - 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하며 유지
 - 최종 CNN 특성 맵은 풀링을 적용해 완전연결층을 통과하도록 크기 조정
 - 바운딩 박스마다 CNN을 돌리는 시간 단축!

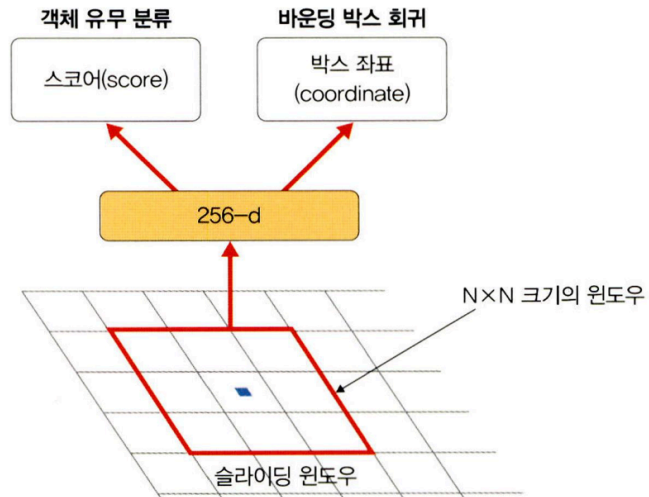


6.2.4 Faster R-CNN

- 정리
 - 더욱 빠른 객체 인식 수행을 위한 네트워크
 - 기존 Fast R-CNN 속도를 느리게 한 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계
 - 후보 영역 추출 네트워크를 추가한 것이 핵심!
 - 외부의 느린 선택적 탐색(CPU로 계산) 대신 내부의 빠른 RPN(GPU로 계산) 사용



- 후보 영역 추출 네트워크

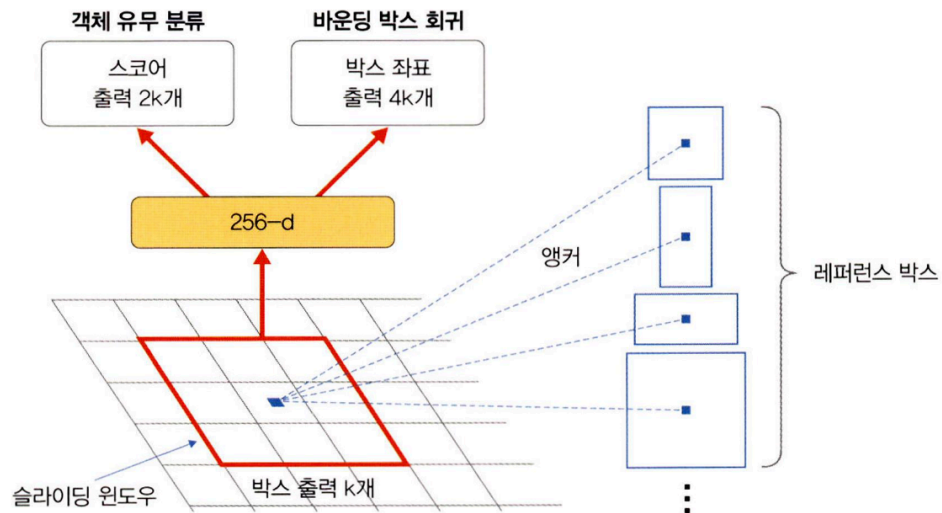


○ 구성

- 특성 맵 크기의 작은 윈도우 영역을 입력으로 받음
- 해당 영역에 객체 존재 유무 판단을 위해 이진 분류를 수행하는 작은 네트워크 생성
- 위치 보정을 위해 바운딩 박스 회귀 추가
- 모든 영역에 대한 객체 존재 유무 확인을 위해 앞서 설계한 작은 윈도우 영역을 이용해 객체 탐색

○ 단점 및 보완

- 객체의 크기와 비율이 다양해 고정된 크기의 입력만으로 다양한 크기와 비율의 이미지를 수용하기 어려움
- 이를 보완하기 위한 방법으로 **앵커** 사용
- 앵커: 여러 크기와 비율의 레퍼런스 박스를 미리 정의하고 각각의 슬라이딩 윈도우 위치마다 출력하도록 설계



2k개의 분류에 대한 출력과 x, y, w, h 위치 보정 값을 위한 4k의 회귀 출력을 가진 특성 맵 크기가 $w \times h$ 라면, 하나의 특성 맵에 앵커가 총 $w \times h \times k$ 개 존재

6.3 이미지 분할을 위한 신경망

- 이미지 분할이란?
 - 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체 추출

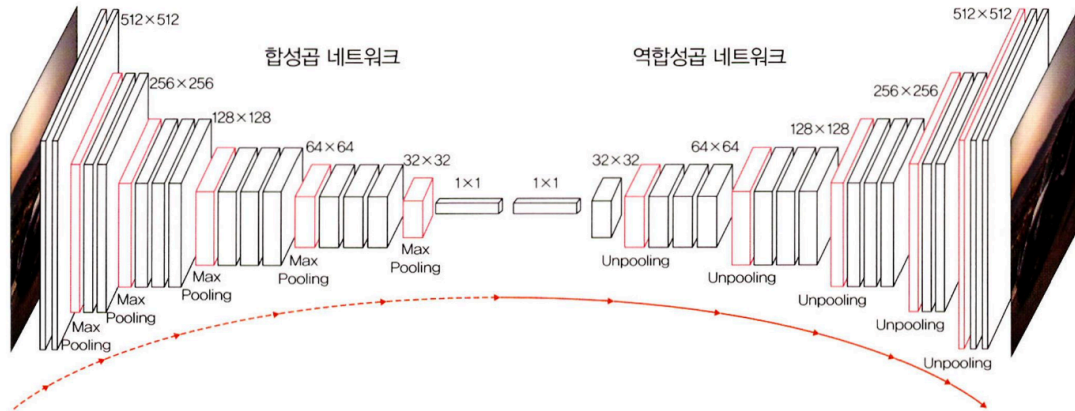
6.3.1 완전 합성곱 네트워크

- 정리
 - 완전연결층 거친 후 위치 정보가 사라진다는 한계 보완
 - 완전연결층을 1×1 합성곱으로 대체
- ⇒ CNN 기반 모델을 변형시켜 이미지 분할에 적합하도록 만든 네트워크

6.3.2 합성곱 & 역합성곱 네트워크

- 등장배경
 - 완전 합성곱 네트워크는 위치 정보도 보존되지만, 다음과 같은 단점이 있음
 - 여러 단계의 합성곱층과 풀링층을 거치며 해상도가 낮아짐
 - 해상도 복원을 위해 업 샘플링 방식을 사용해 이미지의 세부 정보들을 잃어버리는 문제 발생
- ⇒ 이를 해결하기 위해 역합성곱 네트워크 도입

- 정리
 - CNN 최종 출력 결과를 원래 입력 이미지와 같은 이미지로 만들고 싶을 때 사용
 - 시멘틱 분할 등에 활용 가능
 - 역합성곱을 업 샘플링이라고도 함

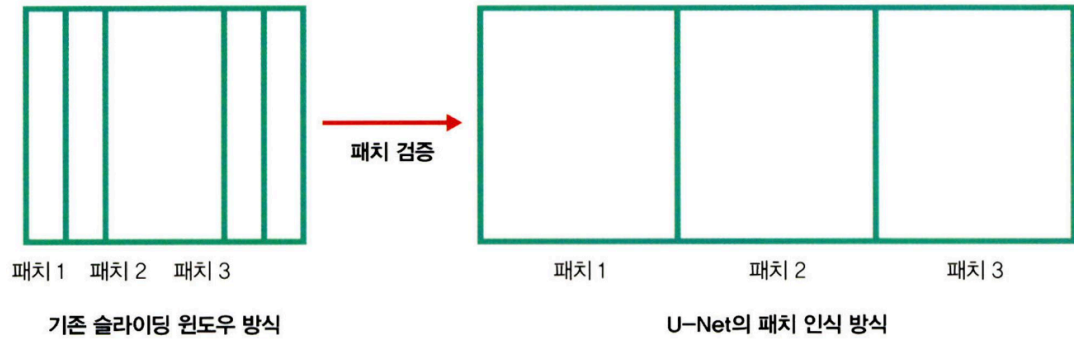


- 동작 방식
 - CNN에서 합성곱층은 합성곱을 사용해 특성 맵 크기를 줄일 때, 역합성곱은 특성 맵 크기를 증가시키는 방식으로 동작
 - 동작 순서
 - 각각의 픽셀 주위에 제로 패딩 추가
 - 패딩된 것에 합성곱 연산 수행

6.3.3 U-Net

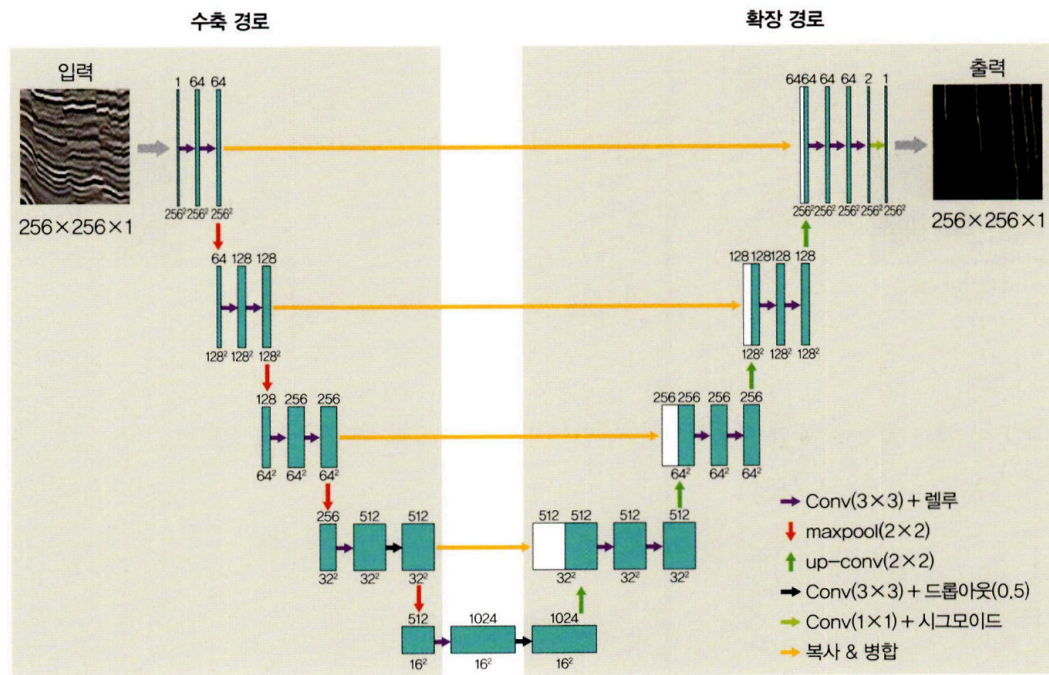
바이오 메디컬 이미지 분할을 위한 합성곱 신경망

- 특징
 - 속도가 빠름



검증이 끝난 패치는 건너뛰기 때문에 속도가 빠름

- 트레이드오프에 빠지지 않음
 - 일반적으로 패치 크기가 커지면 지역화에는 한계가 있음
 - *지역화: 이미지 안 객체 위치 정보를 출력해 주는 것, 주로 바운딩 박스 사용
- 구조
 - FCN 기반으로 구축
 - 수축 경로와 확장 경로로 구성
 - 수축 경로: 특성 맵의 컨텍스트 포착
 - 확장 경로: 특성 맵을 업 샘플링, 수축 경로에서 포착한 컨텍스트와 결합하여 정확한 지역화 수행
 - 각 합성곱 블록이 3x3 합성곱 두 개로 구성되어 있으며, 그 사이에 드롭아웃이 있음
 - 다운 샘플링과 업 샘플링을 순서대로 반복하는 구조



왼쪽의 수축 경로에서의 각 블록은 최대 풀링으로 크기를 줄이며 다음 블록으로 넘어감

오른쪽의 확장 경로에서는 합성곱 블록에 up-conv를 앞에 붙여 수축 과정에서 줄어든 크기를 키워가며 이용

6.3.4 PSPNet

완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈 추가

• 훈련과정

1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링

1×1 , 2×2 , 3×3 , 6×6 크기로 풀링 수행(이때 1×1 크기 맵이 가장 광범위한 정보를 담음)

각각 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담는 것!

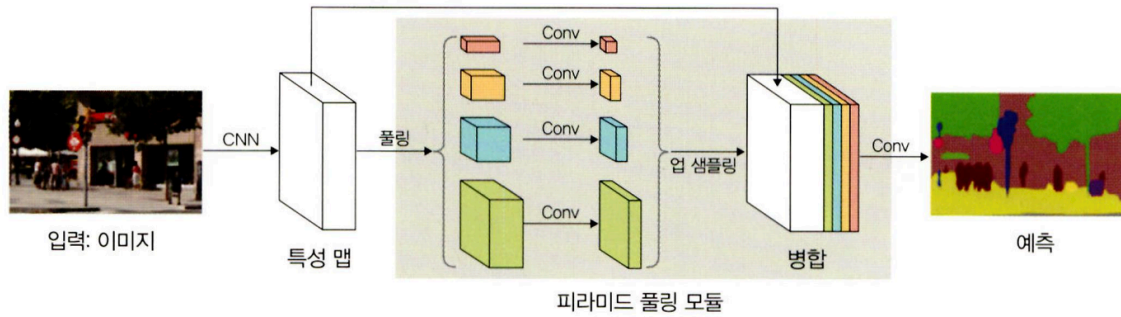
2. 1×1 합성곱을 사용해 채널 수 조정
3. 모듈의 입력 크기에 맞게 특성 맵 업 샘플링

이때 양선형 보간법 사용

***양선형 보간법:**

화소당 선형 보간을 세 번 수행하며, 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치를 곱한 값을 합해 얻음

4. 기존 특성 맵과 1~3 과정에서 생성한 새로운 특성 맵들 병합



여기에서는 풀링 4개를 사용했지만, 구현에 따라 다르게 설정할 수 있음

6.3.5 DeepLabv3/DeepLabv3+

완전연결층의 한계를 극복하기 위해 Atrous 합성곱 사용

- 정리
 - 필터 내부에 빈 공간을 둔 채로 작동(rate: 얼마나 많은 빈 공간을 가질지 결정하는 파라미터, r 이 커질 수록 빈 공간 $\uparrow$)
 - 파라미터 수를 늘리지 않으면서도 수용 영역을 키울 수 있기 때문에 이미지 분할 분야에서 많이 사용됨

*수용 영역

영상 전체 영역에 대해 서로 동일한 중요도를 부여해 처리하는 대신, 특정 범위를 한정해 처리하여 효과적으로 훈련 수행